Федеральное государственное автономное образовательное учреждение высшего образования “Национальный исследовательский университет ИТМО”

Факультет программной инженерии и компьютерных технологий

Лабораторная работа №1

По дисциплине «Тестирование программного обеспечения»

Вариант 3109

Выполнили:

Захарченко Роман Владимирович

Перминов Юрий Константинович

Группа: P3331

Преподаватель: Миняев Илья Андреевич

Санкт-Петербург

2026

Задание

Лабораторная работа #1

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Введите вариант: 3109

1. Функция sec(x)
2. Программный модуль для сортировки массива по алгоритму быстрой сортировки (http://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html)
3. Описание предметной области:

> Минуту спустя последняя планета каталога исчезла, и они снова оказались в реальном мире. Они сидели в приемной, уставленной обитой плюшем мебелью, стеклянными столиками и наградами. Перед ними стоял высокий магратеянин.

Ход выполнения работы

Задание 1

В классе SecSeriesCalculator сумма ряда накапливается по членам, пока модуль следующего члена не станет меньше eps. Дополнительно проверяется корректность входа: x должен быть конечным, eps — положительным и конечным, а область значений ограничена условием |x| ≤ π/3 для устойчивой сходимости.

```
public double sec(double x, double eps) {
    validateInput(x, eps);
    ...
    if (Math.abs(nextTerm) < eps) {
        break;
    }
    sum += nextTerm;
    return sum;
}
```

Математическая основа:

$$\sec x = \sum_{n=0}^{\infty} \frac{(-1)^n E_{2n}}{(2n)!} x^{2n}$$

$$E_{2n} = -\sum_{k=1}^{n} \binom{2n}{2k} E_{2(n-k)},$$

$$\binom{n}{k} \equiv C_n^k = \frac{n!}{k!(n-k)!}$$

Тесты:
Тестирование строилось по классам эквивалентности и граничным значениям. Эталонным значением служило выражение 1 / Math.cos(x).

- Корректные входы: x = 0, малые положительные и отрицательные значения, несколько точек внутри рабочей области $|x| \le \pi/3$.
- Проверка точности: один и тот же x тестируется при разных eps — от очень малого до сравнительно крупного.
- Границы: точка почти на границе, точка ровно на границе и точка сразу за границей допустимой области.
- Невалидные входы: eps = 0, eps < 0, NaN, Infinity; x = NaN, Infinity; значения x вне разрешённой области.
- Математическое свойство: чётность функции sec(-x) = sec(x).

Задание 2

Во второй части требовалось провести модульное тестирование алгоритма быстрой сортировки. Кроме проверки правильности сортировки, в проекте отдельно реализована трассировка выполнения через точки Q1–Q10.

```
public void sort(int[] array, List<String> trace) {
    if (array == null) {
        throw new IllegalArgumentException(...);
    }
    quickSort(array, 0, array.length - 1, trace);
}
```

Тесты:
Здесь использовались два типа тестов: функциональные и trace-тесты. Функциональные тесты отвечают на вопрос «правильно ли отсортирован массив», а trace-тесты — «прошёл ли алгоритм по ожидаемым контрольным точкам».

Пример расчёты трассировок:
Для пояснения возьмём массив [4, 1, 3, 2, 5]. В начале алгоритм заходит в метод quickSort, выбирает средний элемент в качестве pivot (в данном случае 3), после чего запускает разбиение массива. При этом последовательность контрольных точек из теста qstraceNormSortTest получается следующей:

Q1, Q2, Q3, Q4, Q6, Q7, Q5, Q8,

Q1, Q2, Q3, Q4, Q7, Q10,
Q9,
Q1, Q2, Q3, Q4, Q6, Q10,
Q10

1. Q1 и Q2 фиксируют вход в рекурсивную функцию и проверку базового случая.
2. Q3 показывает момент выбора pivot, а Q4 — начало этапа partition.
3. Q6 означает, что правый индекс j был сдвинут влево, потому что справа встретился элемент больше pivot.
4. Q7 соответствует фактическому обмену элементов 4 и 2.
5. Q5 означает сдвиг левого индекса i вправо после того, как слева оказался элемент меньше pivot.
6. Q8 запускает рекурсию для левого подмассива, Q9 — для правого.
7. Q10 ставится при выходе из каждого вызова quickSort.

Именно таким образом trace помогает не только проверить итоговую сортировку, но и доказать, что алгоритм действительно проходит по ожидаемой логике: выбирает pivot, делает обмены, а затем корректно уходит в рекурсию по двум частям.

Задание 3

Третья часть лабораторной работы посвящена выделению сущностей из текстового описания сцены и проверке того, что объектная модель корректно отражает логику происходящего.

- Planet - Простая сущность планеты с проверкой имени.
- Catalog - Каталог планет; умеет хранить список и удалять последнюю планету.
- World - Перечисление состояний мира: CATALOG и REAL_WORLD.
- Person - Базовый персонаж с именем, текущим миром и текущей комнатой.
- Magrathean - Специализация Person; хранит рост и список персонажей, перед которыми он стоит.
- ReceptionRoom - Приёмная комната с мебелью, столиками и наградами.
- SceneService - Сервис, который проигрывает сцену: удаляет планету, переносит персонажей, размещает их в комнате.
- SceneState - Итоговое состояние сцены после выполнения сценария.

Важная особенность модели состоит в том, что она не пытается изобразить предметную область слишком детально. Наоборот, выделены только те сущности и свойства, которые действительно нужны для воспроизведения описанной сцены и для построения тестов.

```
catalog.removeLastPlanet();
for (Person person : people) {
    person.moveToWorld(World.REAL_WORLD);
    person.enterReceptionRoom(receptionRoom);
}
magrathean.standBefore(people);
```

Тестам:

По приложенным отчётам surefire для предметной области все тесты также проходят успешно: 3 теста в CatalogTest, 3 теста в DomainValidationTest и 4 теста в SceneServiceTest. Таким образом, в domain-модуле закрыты и позитивные, и негативные сценарии. Численные метрики jacoco для этих классов в приложенном архиве отсутствуют, поэтому в отчёте корректнее ограничиться качественным выводом о достаточности покрытия, а не придумывать точные проценты.

Вывод

В ходе лабораторной работы было проведено модульное тестирование трёх различных по характеру частей проекта. В модуле sec было проверено вычисление математической функции через степенной ряд с учётом точности, границ области определения и обработки некорректных входов. В модуле quicksort были протестированы как функциональные свойства сортировки, так и внутренняя логика работы алгоритма с помощью трассировки Q1–Q10. В модуле предметной области была построена и проверена компактная объектная модель сцены, включая валидацию данных и интеграционный сценарий.

По приложенным отчётам все тесты проходят успешно, а для модулей sec и quicksort показатели покрытия находятся на высоком уровне. Это позволяет сделать вывод, что реализованное тестовое покрытие для учебной задачи является достаточным: оно охватывает корректные и ошибочные входы, граничные случаи, основные ветви выполнения и ключевые сценарии взаимодействия объектов.